

# Systemy czasu rzeczywistego, Projekt 4

Piotr Gugnowski, wydział EiTI, nr indeksu 320690

Czerwiec 2026

## Zadanie 4.1 — System CAN: suma kontrolna

### Treść zadania

Jak wiemy, determinizm jest bardzo ważnym, ale nie jedynym i często nie najważniejszym wymaganiem stawianym sieciom czasu rzeczywistego. W praktyce ogromną rolę odgrywa bezpieczeństwo przesyłanych informacji. Pod tym pojęciem rozumiemy w węższym sensie zgodność informacji nadawanej i odbieranej w sieci. W szerszym sensie bezpieczeństwo informacji przesyłanych w sieci polega również na ich odpowiednim zabezpieczeniu w celu utrudnienia lub uniemożliwienia dostępu do nich stronom trzecim, lub/i zastosowaniu odpowiednich technik kryptograficznych. W tym zadaniu interesować nas będzie wyłącznie bezpieczeństwo w zdefiniowanym wyżej węższym sensie. Jak wiadomo, na skutek zakłóceń komunikacyjnych np. wskutek oddziaływania silnych zaburzeń elektromagnetycznych, przesyłana informacja może ulec zniekształceniu. Dla celów detekcji błędów transmisji stosowane są różne techniki. Jedną z nich jest stosowana dość powszechnie w profesjonalnych sieciach komunikacyjnych redundantna informacja zwana sumą kontrolną lub cykliczną sumą redundancyjną. Suma kontrolna jest tworzona według określonego algorytmu na podstawie wysyłanych informacji przez nadajnik. Suma ta dołączana jest do strumienia wysyłanej informacji. Odbiornik tworzy własną sumę kontrolną według tego samego algorytmu co nadajnik, ale na podstawie strumienia danych odbieranych. Różnica w odebranej i zrekonstruowanej w odbiorniku sumie kontrolnej wskazuje na wystąpienie błędu lub błędów komunikacyjnych i pozwala na podjęcie odpowiednich akcji. Należy jednak pamiętać, że:

- istnieje skończone prawdopodobieństwo zgodności sum kontrolnych odebranej i utworzonej w odbiorniku mimo wystąpienia błędu lub błędów transmisji;
- prawdopodobieństwo to jest w profesjonalnych systemach sieciowych znikome i minimalizowane dzięki odpowiedniemu doborowi algorytmów wyznaczania sumy kontrolnej i starannemu doborowi wielomianów generacyjnych.

W przypadku sieci CAN stosowany jest następujący algorytm.

### Algorytm wyznaczania CRC

Niech  $CRC\_RG(14:0)$  oznacza rejestr przechowujący wartość sumy kontrolnej CRC, natomiast  $NXTBIT$  oznacza kolejny bit transmitowanego ciągu bitów poczynawszy od SOF i skończywszy na ostatnim bicie pola DATA. Suma CRC obliczana jest następująco:

```

CRC_RG = 0;
REPEAT
    CRCNXT = NXTBIT EXOR CRC_RG(14)
    CRC_RG(14:1) = CRC_RG(13:0);
    CRC_RG(0) = 0;
    IF CRCNXT THEN
        CRC_RG(14:0) = CRC_RG(14:0) EXOR (4599hex);
    ENDIF
UNTIL (zostanie osiągnięte CRC SEQUENCE lub wystąpi błąd transmisji)

```

Napisać aplikację wyznaczającą sumę kontrolną dla sieci CAN. Aplikacja ta ma być optymalizowana ze względu na minimalizację czasu realizacji algorytmu wyznaczania CRC.

### Założenia:

- Aplikacja ma umożliwiać wprowadzenie z klawiatury ciągu do 96 bitów.
- Aplikacja powinna wyznaczyć sumę kontrolną z wprowadzonego ciągu bitowego i wyświetlić ją w postaci heksadecymalnej.
- Aplikacja powinna mieć możliwość deklarowania liczby powtórzeń obliczenia sumy kontrolnej od 1 do  $10^9$ .
- Aplikacja powinna mieć możliwość pomiaru i wyświetlania łącznego czasu realizacji zadeklarowanej liczby powtórzeń obliczenia sumy kontrolnej oraz średniego czasu jednokrotnej realizacji obliczenia CRC.
- Aplikacja powinna być dostarczona w postaci kodu maszynowego wykonywalnego.

## Implementacja

Kod źródłowy znajduje się w pliku `src/main.cpp`.

### Reprezentacja wejścia

Ciąg bitów wczytany z klawiatury jest pakowany w strukturę `BitInput`: 16 bajtów przechowuje bity MSB-first (pierwszy znak  $\rightarrow$  bit 7 bajtu 0, drugi  $\rightarrow$  bit 6 bajtu 0 itd.), a pole `nbits` przechowuje dokładną liczbę bitów (0–96).

### Tablica przeglądowa generowana w czasie kompilacji

Funkcja `can_crc15()` oblicza CRC-15/CAN w dwóch fazach:

- Bajt po bajcie** (dla  $\lfloor n/8 \rfloor$  pełnych bajtów) przy użyciu 256-elementowej tablicy `TABLE` (512 B, mieści się w L1 cache). Klucz do tablicy: `key = ((crc » 7) ^ byte) & 0xFF`; aktualizacja: `crc = ((crc « 8) & 0x7FFF) ^ TABLE[key]`. Daje 8-krotną redukcję liczby iteracji względem przetwarzania bit po bicie.
- Pozostałe bity** (0–7, ostatni niepełny bajt): iteracja bit po bicie z bezgałęziowym krokiem  
`crc ^= crcnxt * 0x4599u` (eliminacja branch misprediction).

Tablica jest obliczana jako `constexpr`.

## Zabezpieczenia integralności benchmarku

- `volatile uint16_t sink` — kompilator nie może wynieść wywołania funkcji poza pętlę (efekt uboczny obserwowalny z zewnątrz).
- `__attribute__((noinline))` na funkcji `can_crc15` — blokuje inlining, przez co kompilator nie widzi wnętrza funkcji i nie może udowodnić stałości wyniku.

## Pełny kod funkcji CRC

```
1 uint16_t can_crc15(const BitInput& in) noexcept {
2     uint16_t crc = 0;
3     const int full = in.nbits / 8;
4     const int rem = in.nbits % 8;
5
6     for (int i = 0; i < full; ++i) {
7         const uint8_t key =
8             static_cast<uint8_t>(((crc >> 7) ^ in.bytes[i]) & 0xFFu);
9         crc = static_cast<uint16_t>(
10             ((crc << 8) & 0x7FFFu) ^ TABLE.v[key]);
11     }
12
13     if (rem) {
14         const uint8_t last = in.bytes[full];
15         for (int i = 7; i >= 8 - rem; --i) {
16             const uint8_t crcnxt = static_cast<uint8_t>(
17                 ((crc >> 14) ^ ((last >> i) & 1u)) & 1u);
18             crc = static_cast<uint16_t>((crc << 1) & 0x7FFFu);
19             crc ^= static_cast<uint16_t>(crcnxt * 0x4599u);
20         }
21     }
22     return crc;
23 }
```

## Wyniki i pomiary czasu

Pomiary wykonano na Apple Silicon (ARM64), kompilacja: `clang++ -O3 -march=native`,  $n = 10^9$  powtórzeń.

| Wejście             | Bity | CRC-15/CAN | Czas łączny | Czas/iter. |
|---------------------|------|------------|-------------|------------|
| 96 zer (00...0)     | 96   | 0x0000     | 319,5 ms    | 0,320 ns   |
| 96 jedynek (11...1) | 96   | 0x072A     | 313,7 ms    | 0,314 ns   |

Tabela 1: Wyniki benchmarku CRC-15/CAN dla  $n = 10^9$  iteracji.

Wynik dla 96 zer (0x0000) jest zgodny z oczekiwaniami: wszystkie bity wejściowe zerowe powodują, że CRCNEXT jest zawsze zerem, rejestr CRC pozostaje zerowy przez cały czas obliczeń.

## Zadanie 4.2 — System CAN: identyfikatory wiadomości

### Treść zadania

Jak wiadomo, węzły CAN są generatorami wiadomości. Każda wiadomość ma swój własny identyfikator. Zależnie od specyfikacji sieci CAN występują w specyfikacji standardowej identyfikatory 11-bitowe, a w specyfikacji rozszerzonej identyfikatory 29-bitowe. W związku z tym teoretycznie możliwe jest zaadresowanie maksymalnie odpowiednio  $2^{11}$  lub  $2^{29}$  wiadomości.

Czy tak jest w rzeczywistości? Proszę podać istniejące ograniczenia w specyfikacji CAN (o ile występują).

### Odpowiedź

Nie, nie wszystkie kombinacje identyfikatorów są dostępne. Specyfikacja CAN zabrania używania identyfikatorów, których **pierwsze 7 bitów** to same jedynki (1111111...). Taka kombinacja wygląda na magistrali identycznie jak sygnał błędu pasywnego (*passive error flag* - 6 bitów recessive) lub jak przerwa między ramkami (IFS), co mogłoby wprowadzić odbiorniki w błąd.

| Format      | Bity ID | Zakazane ID            | Dostępne ID                                            |
|-------------|---------|------------------------|--------------------------------------------------------|
| Standardowy | 11      | $2^4 = 16$             | $2048 - 16 = \mathbf{2032}$                            |
| Rozszerzony | 29      | $2^{22} = 4\,194\,304$ | $536\,870\,912 - 4\,194\,304 = \mathbf{532\,676\,608}$ |

Tabela 2: Rzeczywista liczba dostępnych identyfikatorów w sieci CAN.

W praktyce ograniczenie to jest nieistotne, typowa sieć przemysłowa korzysta z kilkudziesięciu do kilkuset typów wiadomości, co stanowi ułamek procenta dostępnej przestrzeni adresowej.

## Zadanie 4.3 — System CAN: rzeczywisty czas trwania transmisji ramki

### Treść zadania

Dokładna analiza czasu transmisji ramki CAN wskazuje, że przy transmisji tej samej liczby bajtów informacji w polu danych czas trwania transmisji jest różny. Czas ten zależy od kontekstu. Przyczyna leży w sposobie kodowania informacji, a w szczególności w zastosowanej technice synchronizacji transferów sieciowych wspieranych przez dodatkowe bity synchronizujące (technika *bit stuffing*).

#### Założenia:

- Prędkość transmisji w sieci CAN: 1 Mb/s.
- Przesyłamy w polu danych 8 bajtów informacji.
- Identyfikator wiadomości podlega swobodnemu wyborowi z ew. ograniczeniami (por. zadanie 4.2).
- Stosujemy ramkę o identyfikatorze 29-bitowym.
- Współczynnik efektywności transmisji zdefiniowany jako stosunek liczby bitów wysłanej informacji (64 bity) do całkowitej liczby bitów ramki (łącznie z bitami przerwy między ramkami i bitami dodatkowymi dodanymi przez operację *bit stuffing*).

- a) Wyznaczyć najkrótszy czas transmisji całej ramki CAN. Podać przykład zawartości pola danych, dla której ma to miejsce. Podać wartość współczynnika efektywności transmisji.
- b) Wyznaczyć najdłuższy czas transmisji całej ramki CAN. Podać przykład zawartości pola danych, dla której ma to miejsce. Podać wartość współczynnika efektywności transmisji.

## Wyprowadzenie liczby bitów ramki

Rozszerzona ramka CAN 2.0B z 8 bajtami danych składa się z następujących pól:

| Pole                   | Bitów      |
|------------------------|------------|
| SOF                    | 1          |
| Arbitration (ID+flagi) | 32         |
| Control                | 6          |
| Data                   | 64         |
| CRC sequence           | 15         |
| CRC delimiter          | 1          |
| ACK slot               | 1          |
| ACK delimiter          | 1          |
| EOF                    | 7          |
| IFS                    | 3          |
| <b>Łącznie</b>         | <b>131</b> |

$$L_{\text{stały}} = 1 + 32 + 6 + 64 + 15 + 1 + 1 + 1 + 7 + 3 = \mathbf{131} \text{ bitów}$$

Technika **bit stuffing** polega na automatycznym wstawianiu przez nadajnik jednego bitu o przeciwnej wartości po każdym 5 kolejnych bitach tej samej wartości. Odbiornik usuwa te dodatkowe bity. Cel: utrzymanie synchronizacji zegara odbiornika, który nie ma osobnego sygnału taktowania i synchronizuje się na zmianach stanu na magistrali.

Bit stuffing obowiązuje dla regionu: SOF + Arbitration + Control + Data + CRC sequence:

$$N_{\text{fill}} = 1 + 32 + 6 + 64 + 15 = \mathbf{118} \text{ bitów}$$

Przy prędkości 1 Mb/s czas jednego bitu wynosi  $T_b = 1 \mu\text{s}$ .

### a) Najkrótszy czas transmisji

Najkrótszy czas uzyskuje się przy **zerowej liczbie bitów stuffujących**, co zachodzi gdy w regionie 118 bitów nie pojawi się żadna sekwencja 5 kolejnych bitów tej samej wartości.

**Przykład danych spełniających ten warunek:**

0x55 0x55 0x55 0x55 0x55 0x55 0x55 0x55

Bajt 0x55 = 01010101<sub>2</sub> — bity naprzemienne, nie tworzą żadnej sekwencji 5 identycznych. Identyfikator musi mieć analogiczną właściwość: żaden fragment jego 29 bitów nie może tworzyć serii 5 identycznych, a ponadto złożenie SOF + ID + pola Control (łącznie z wynikową sumą CRC) nie może stworzyć takiej serii na granicy sąsiednich pól. W praktyce identyfikator z naprzemiennymi bitami (postaci 01010... lub 10101...) spełnia ten warunek przy naprzemiennych danych 0x55.

$$L_{\text{min}} = L_{\text{stały}} + N_{\text{stuff}} = 131 + 0 = \mathbf{131} \text{ bitów}$$

$$T_{\min} = 131 \cdot 1 \mu s = \mathbf{131 \mu s}$$

**Współczynnik efektywności transmisji** (stosunek bitów użytecznych do wszystkich bitów ramki):

$$\eta_{\max} = \frac{64}{131} \approx \mathbf{48,9\%}$$

## b) Najdłuższy czas transmisji

Najdłuższy czas uzyskuje się przy **maksymalnej liczbie bitów stuffujących**. Bit stuffujący jest wstawiany po każdym 5 bitach tej samej wartości, więc najgorszy wzorzec to grupy pięciu identycznych bitów: 11111 00000 11111 00000...

Dla regionu 118 bitów każda grupa 5 bitów wymusi wstawienie 1 bitu stuffującego:

$$N_{\text{stuff,max}} = \left\lfloor \frac{118}{5} \right\rfloor = \mathbf{23}$$

**Przykład danych generujących dużo stuffowania:**

0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00

64 kolejne zera w polu danych samo w sobie daje  $\lceil 64/5 \rceil = 13$  bitów stuffujących. Aby osiągnąć 23 bity stuffujące łącznie, identyfikator powinien zawierać długie serie identycznych bitów (np. postaci 11111 00000...), a wynikowa suma kontrolna CRC — obliczana automatycznie z ID i danych, bez możliwości swobodnego wyboru — musi wspólnie z ID wypełnić pozostałe  $23 - 13 = 10$  bitów stuffujących w regionie poza polem danych. Znalezienie wartości ID spełniającej ten warunek wymaga iteracyjnego sprawdzania kandydatów z uwzględnieniem wynikowego CRC.

$$L_{\max} = L_{\text{stały}} + N_{\text{stuff,max}} = 131 + 23 = \mathbf{154 \text{ bitów}}$$

$$T_{\max} = 154 \cdot 1 \mu s = \mathbf{154 \mu s}$$

**Współczynnik efektywności transmisji:**

$$\eta_{\min} = \frac{64}{154} \approx \mathbf{41,6\%}$$

## Podsumowanie

| Scenariusz       | Dane (8 B) | Bity stałe | Bity stuff. | Łącznie | Wsp. ef. trans. $\eta$ |
|------------------|------------|------------|-------------|---------|------------------------|
| Najkrótsza ramka | 0x55×8     | 131        | 0           | 131     | 48,9%                  |
| Najdłuższa ramka | 0x00×8     | 131        | 23          | 154     | 41,6%                  |